

# The DOM, Events & Logic

Turning a static page into something that reacts — select elements, respond to the user, and control what your code does.

## WHAT WE COVER

[The DOM](#)[Event listeners](#)[Conditionals](#)[switch](#)[Loops](#)

You can already build and style a page. Today it comes alive: JavaScript reaches into the HTML, listens for what the user does, and decides what happens next. By the end you'll wire up a working interactive component from scratch.

# The DOM, Events & Logic

JavaScript has two superpowers on a page: it can *reach into* your HTML and change it, and it can *react* to what the user does. Everything below builds toward one idea — connecting user actions to changes on the screen.

## ON THIS HANDOUT

01. The DOM: your page as objects
02. Selecting elements
03. Changing content, styles & classes
04. Creating & removing elements
05. Events & event listeners
06. The event object, common events & delegation
07. Logic & flow control
08. Loops

## 01 The DOM: your page as objects

When the browser loads your HTML, it doesn't just draw it — it builds a live **tree of objects** from your tags. That tree is the **DOM** (Document Object Model).

- ▶ Every tag ( `<div>` , `<p>` , `<button>` ) becomes a **node** you can read and change.
- ▶ Change the DOM and the page updates **instantly** — no reload. That's what makes sites feel alive.
- ▶ Your entry point into the whole tree is the global `document` object.

Think of `document` as the root. From it you can reach any element on the page.

## 02 Selecting elements

Before you can change something, you have to grab it. Modern code uses `querySelector` , which takes the **same selectors you already know from CSS**.

```
document.getElementById("title"); // by id -> one element
document.querySelector(".btn");   // FIRST match of a CSS selector
document.querySelectorAll("li");  // ALL matches -> a NodeList
document.querySelector("#nav .link.active"); // any CSS selector works
```

JS

- ▶ `querySelector` returns the **first** match (or `null` if none).

- ▶ `querySelectorAll` returns a **list** you can loop over.
- ▶ Prefer these two — they can do everything the older `getElementBy*` methods do.

## 03 Changing content, styles & classes

### Content

```
const box = document.querySelector(".box");

box.textContent = "Just text";    // safe: sets plain text
box.innerHTML  = "<b>Bold</b>";    // parses HTML (careful with user input)
box.value      = "hi";           // for <input>, <textarea>, <select>
```

JS

### Styles & classes

Prefer toggling **classes** over setting inline styles — keep the look in your CSS file.

```
box.style.color = "blue";        // inline style (use sparingly)

box.classList.add("active");     // add a class
box.classList.remove("active");  // remove a class
box.classList.toggle("open");    // add if missing, remove if present
box.classList.contains("active"); // -> true / false
```

JS

## 04 Creating & removing elements

You can build new nodes in JavaScript and drop them into the page — this is how lists, cards and messages appear dynamically.

```
const li = document.createElement("li"); // make a new node
li.textContent = "New item";

list.appendChild(li); // add as the last child
list.prepend(li);     // add as the first child
li.remove();           // delete it from the page
```

JS

## 05 Events & event listeners

An **event** is something that happens on the page: a click, a key press, a form submit, the mouse moving. You **listen** for an event and run a function — the **handler** — when it fires.

```
const btn = document.querySelector("#myBtn");

btn.addEventListener("click", () => {
  alert("Button clicked!");
});

// a named handler can be removed later
function onClick() { console.log("hi"); }
btn.addEventListener("click", onClick);
btn.removeEventListener("click", onClick);
```

JS

- ▶ `addEventListener(type, handler)` is the modern, flexible way to react to events.
- ▶ You can attach many listeners to the same element — they don't overwrite each other.

## 06 The event object, common events & delegation

### The event object

Every handler receives an **event object** describing what happened.

```
input.addEventListener("input", (e) => {
  console.log(e.target);      // the element that fired the event
  console.log(e.target.value); // its current value
});

form.addEventListener("submit", (e) => {
  e.preventDefault();        // stop the page from reloading
});
```

JS

### Common events

- ▶ **Mouse:** `click`, `dblclick`, `mouseover`, `mouseout`
- ▶ **Keyboard:** `keydown`, `keyup`
- ▶ **Form:** `input`, `change`, `submit`, `focus`, `blur`
- ▶ **Page:** `DOMContentLoaded`, `load`, `scroll`

### Event delegation

Clicks **bubble up** through parents. So instead of adding a listener to every child, add **one** to the parent — perfect for lists that grow.

```
document.querySelector("#list").addEventListener("click", (e) => {
  if (e.target.matches("li")) {
    e.target.classList.toggle("done");
  }
});
```

JS

## 07 Logic & flow control

### Operators

- **Compare:** `===` (value + type), `!==`, `>`, `<`, `>=`, `<=`. Always prefer `===` over `==`.
- **Logical:** `&&` (and), `||` (or), `!` (not).
- **Falsy values:** `false`, `0`, `""`, `null`, `undefined`, `NaN` — everything else is truthy.

### if / else if / else

```
if (score >= 90) {
  grade = "A";
} else if (score >= 75) {
  grade = "B";
} else {
  grade = "C";
}
```

JS

### Ternary & shortcuts

```
let status = age >= 18 ? "Adult" : "Minor"; // condition ? ifTrue : ifFalse
let name   = userName || "Guest";         // fallback if left side is falsy
let city   = user?.city ?? "N/A";         // fallback only on null / undefined
```

JS

### switch — one value, many options

```
switch (day) {
  case "Sat":
  case "Sun":
    console.log("Weekend");
    break;
  case "Mon":
    console.log("Back to work");
    break;
  default:
    console.log("A weekday");
}
```

JS

Cases “fall through” without `break` — handy for grouping (Sat/Sun above), a bug everywhere else. Don't forget it.

## 08 Loops

Loops repeat a block so you don't copy-paste — ideal for walking through lists of data or DOM elements.

## for & while

```

for (let i = 0; i < 5; i++) {
  console.log(i);           // 0 1 2 3 4
}

let n = 0;
while (n < 3) {              // check first, then run
  console.log(n);
  n++;
}

```

JS

## Looping over collections

```

const colors = ["red", "green", "blue"];

for (const c of colors) { console.log(c); } // for...of -> values
colors.forEach((c, i) => console.log(i, c)); // forEach with index

const user = { name: "Abhi", age: 25 };
for (const key in user) { console.log(key); } // for...in -> keys

```

JS

## break & continue

```

for (let i = 0; i < 10; i++) {
  if (i === 5) break;        // stop the loop entirely
  if (i % 2 === 0) continue; // skip to the next iteration
  console.log(i);           // 1 3
}

```

JS

## Putting it together: a live counter

DOM + event + logic in one tiny example — the pattern behind most interactive UI.

```

let n = 0;
const out = document.getElementById("count");

document.getElementById("add").addEventListener("click", () => {
  n++; // logic
  out.textContent = n; // change the DOM
  out.classList.toggle("high", n >= 10); // react past a threshold
});

```

JS

### QUICK RECAP

The **DOM** lets JS select and change elements · **events** connect user actions to your code · **if / switch** decide what runs · **loops** repeat work over data. Together, they turn a static page into an app.